

Engineering Reliable LLM-Based Data Analysis: An Empirical Study of Schema Grounding, Planning, Verification, and SQL Repair

Author: Mevin Jose

Date: August 2026

Status: Pre-print & Submission-Ready Manuscript

Primary Source of Truth: ``docs/research_paper/latex/main.tex`` (This Markdown document is a synchronized non-authoritative companion)

Artifact Package: ``docs/research_paper/`` (Figures 1–7, LaTeX Tables, ``macros.tex``)

Frozen Dataset Hash: SHA-256 ``0c9807d5867ff9cb6a9252437dab31660b62b2e6c9d09c5e54b1dfc7edc43e04``

Frozen Database Hash: SHA-256 ``8550c4cc6d670aa0441bc898e47a57a40001858fc3f13dc5cb16fb90ca11c130``

***Note on Authority:** The LaTeX manuscript (``docs/research_paper/latex/main.tex``) is the single publication source of truth. This Markdown document is maintained as a synchronized companion.*

Abstract

While Large Language Models (LLMs) demonstrate notable code-generation capabilities, translating natural language questions into reliable analytical SQL over relational databases (Text-to-SQL) remains brittle in practice. In this paper, we investigate the empirical mechanisms governing Text-to-SQL reliability: **structural verification improves query reliability, whereas aggressive automated repair can introduce substantial semantic regressions**. We evaluate a multi-stage reliability pipeline on a frozen 500-query benchmark across 8 business domains over a public relational e-commerce data warehouse (the Olist dataset, 9 tables, 100,000+ orders). Our system achieves a **73.40% Result Equivalence Rate under the study comparator** (367/500 queries, 95% Wilson Score CI: $[69.26\%, 77.18\%]$, Clopper-Pearson Exact CI: $[69.30\%, 77.22\%]$), **31.00% Exact Match**, and **100.00% SQL Execution Success** with a mean latency of 64.04s (\$p50\$: 56.47s, \$p95\$: 121.92s). In a controlled 4-way 100-query ablation, activating deterministic Abstract Syntax Tree (AST) structural verification significantly improves result equivalence over unverified planning (Config C 26.0% vs. Config B 15.0%, McNemar exact $p=0.0192$, $\text{Odds Ratio}=3.75$). However, an exhaustive audit of all 101 repair events reveals that automated self-repair is a double-edged mechanism: while 97 of 101 post-repair queries (96.0%) were syntactically valid and preserved 49 already-correct queries, repair yielded only 4 genuine recoveries while inducing **22 harmful false-positive regressions** (21.8% of repair events) where previously correct queries were degraded. Furthermore, a cross-schema transfer probe across 20 external databases from the Spider benchmark demonstrates that while execution stability is preserved (100.0%), result equivalence drops to **18.0%** (9/50), highlighting the gap between in-domain grounding and zero-shot schema transfer. We conclude that conservative, uncertainty-aware structural verification is preferable to unconstrained automated repair loops.

1. Introduction and Central Thesis

Natural language interfaces to relational databases (Text-to-SQL) promise to democratize data analytics for non-technical stakeholders. However, evaluating LLM-generated SQL against complex, multi-table schemas reveals a foundational insight:

Executable SQL is not necessarily reliable analytical SQL.

A query may compile cleanly and execute without database runtime exceptions, yet return subtly corrupted figures due to systemic structural failure modes: schema hallucinations, join-path omissions, aggregation grain mismatches, and filter inconsistencies.

Central Thesis

Rather than presenting Text-to-SQL as an unconstrained agentic pipeline, this empirical study establishes that:

Deterministic structural verification improves Text-to-SQL reliability, while aggressive automated repair can introduce semantic regressions.

Our evidence directly supports this thesis:

1. **Verification Benefit:** In a controlled ablation, activating AST-based structural verification improves result equivalence from **15.0% to 26.0%** over unverified planning (McNemar exact $p=0.0192$, $\text{OR}=3.75$).

2. **Repair Hazard:** An audit of all 101 repair events reveals that compiler execution validity masks semantic degradation, producing **22 harmful false-positive regressions** against only **4 genuine recoveries**.

2. Research Questions and Summary of Findings

• **RQ1: Overall Reliability** --- Does the multi-stage pipeline achieve high result equivalence on a complex data warehouse?

\$\rightarrow\$ **Answer:** On the 500-query benchmark across 8 domains, the system achieves **73.40% Result Equivalence** under the study comparator (95% Wilson CI: $[69.26\%, 77.18\%]$) and **100.00% SQL Execution Success**.

• **RQ2: Verification Impact** --- What is the marginal effect of deterministic AST structural verification?

\$\rightarrow\$ **Answer:** Adding AST structural verification increases execution success from 34.0% to 65.0% and result equivalence from 15.0% to 26.0%; the matched comparison for result equivalence is statistically significant (McNemar exact $p=0.0192$, $\text{OR}=3.75$).

• **RQ3: Repair Dynamics** --- Does automated self-repair reliably fix broken queries without harming valid ones?

\$\rightarrow\$ **Answer:** No. While 96.0% of post-repair queries execute cleanly and 49 correct queries are maintained, repair produced only **4 genuine recoveries** while causing **22 false-positive regressions**.

• **RQ4: Failure Taxonomy** --- What structural failure modes dominate remaining non-equivalent queries?

\$\rightarrow\$ **Answer:** Among the 133 non-equivalent queries, failure is dominated by missing join paths (27.8%), filter omissions/errors (24.8%), and aggregation mismatches (24.1%).

• **RQ5: Cross-Schema Transfer** --- Does warehouse-grounded reliability transfer to unseen external schemas?

\$\rightarrow\$ **Answer:** Execution stability is fully preserved (100.0%), but result equivalence drops to **18.0%** (9/50) on a 20-database Spider transfer probe, exposing the gap between deep schema grounding and zero-shot transfer.

3. Summary of Contributions

- Controlled Empirical Decomposition:** We evaluate the marginal effect of deterministic AST verification over unverified planning, demonstrating a statistically significant reliability gain for result equivalence ($p=0.0192$).
- Repair-Risk Characterization:** We present an exhaustive 4-way semantic audit of 101 repair events, showing that execution-valid repair can induce substantial false-positive regressions (22 harmed vs. 4 recovered).
- Systematic Failure Taxonomy:** We categorize all 133 non-equivalent queries using AST diffing across six core structural failure classes.
- Cross-Schema Transfer Probe:** We report zero-shot transfer evaluation across 20 unseen SQLite databases from the Spider benchmark, demonstrating execution robustness alongside a marked semantic generalization drop.
- Audited Open Science Package:** We release all source code, frozen benchmark definitions, evaluation scripts, LaTeX manuscripts, and a cryptographic SHA-256 artifact manifest.

4. Related Work and Positioning

- **Text-to-SQL Decomposition & Benchmarks:** Benchmarks such as Spider (Yu et al., 2018) and BIRD (Li et al., 2023) evaluate LLMs on multi-table joins and nested subqueries. Decomposition frameworks such as DIN-SQL (Pourreza & Rafiei, 2023), MAC-SQL (Wang et al., 2024), and CHESS (Talaie et al., 2024) show that dividing SQL generation into sub-problems improves performance over monolithic prompting. Our work builds upon this paradigm by introducing deterministic, AST-level structural verification gates that inspect query structure prior to execution.
- **Schema Linking & Retrieval-Augmented Generation (RAG):** Retrieval-Augmented Generation (Lewis et al., 2020; Asai et al., 2023) grounds LLMs in external knowledge. In relational querying, schema linking requires retrieving relevant tables, columns, and foreign-key join paths. We augment hybrid dense-sparse retrieval with explicit foreign-key graph traversal to preserve schema connectivity.
- **Structural Verification vs. Execution-Guided Self-Repair:** Iterative reasoning architectures such as ReAct (Yao et al., 2022) and Reflexion (Shinn et al., 2023) leverage feedback loops for self-correction. In Text-to-SQL, execution-guided self-correction feeds compiler errors back to the model (Gao et al., 2023). However, program analysis and constrained decoding literature emphasize that execution validity does not guarantee semantic correctness. Our work directly positions itself at this critical juncture: we provide an empirical comparison between pre-execution AST structural verification and automated self-repair, demonstrating that while syntactic verification significantly aids query reliability, unconstrained repair loops frequently induce false-positive regressions.

5. Reliability-Oriented System Architecture

Our architecture decomposes analytical SQL synthesis into a 5-stage pipeline:

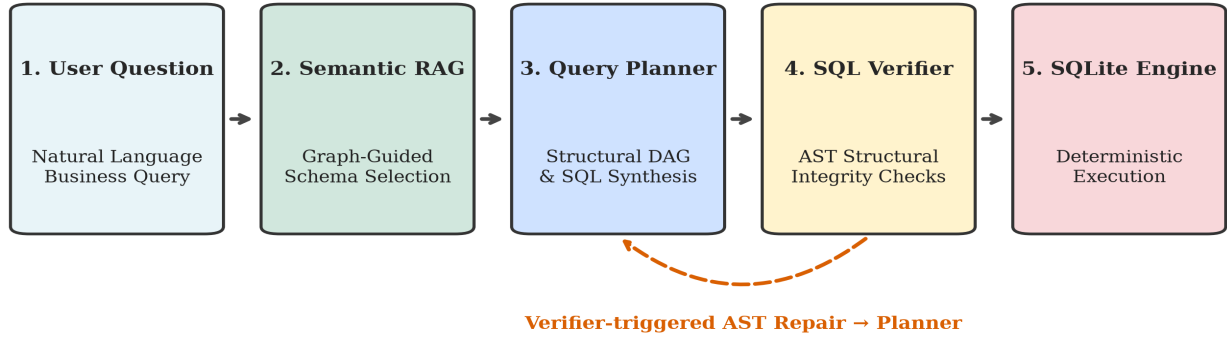


Figure 1: Reliability-Oriented Multi-Stage Architecture with Structural Verification & Repair.

5.1 Graph-Guided Semantic Schema RAG

Rather than passing an entire database schema into the LLM context, our retriever combines BM25 keyword matching, dense embedding retrieval via SentenceTransformers, and foreign-key graph traversal. Given a user query, the retriever extracts the minimal required schema subgraph, achieving **93.07% Table Precision** and **95.33% Table Recall** (Table Exact Match: 82.60%) across the 500-query benchmark.

5.2 Structured DAG Query Planning & Deterministic Plan Validation

Before emitting SQL code, the planner synthesizes a structural JSON DAG declaring target metrics, grain dimensions, required tables, and explicit join paths. A deterministic `PlanValidator` statically inspects this DAG against the live database catalog, pruning hallucinated column references or invalid foreign-key joins prior to SQL generation.

5.3 AST-Based SQL Structural Verification & Closed-Loop Repair

Generated SQL statements are parsed into Abstract Syntax Trees (AST) using SQLGlot. The `SQLSemanticVerifier` statically enforces:

- **Dialect Compliance:** Translating dialect-specific functions (`DATEDIFF`, `DATE\_TRUNC`) into canonical SQLite expressions (`strftime`, `julianday`).
- **Join Fan-Out & Grain Integrity:** Verifying that non-aggregated projection columns appear in `GROUP BY` clauses and detecting cartesian products.
- **Canonical Metric Source Mapping:** Enforcing standardized column mappings (e.g., mapping revenue to `order\_items.price`).

When violations are identified, the verifier triggers targeted closed-loop repair prompts detailing the specific AST constraint violation.

6. Experimental Methodology

6.1 Data Warehouse & Benchmark Corpus

We evaluate our system on a multi-table relational e-commerce schema constructed from the Brazilian E-Commerce Public Dataset (Olist):

- **9 Relational Tables:** `customers`, `orders`, `order\_items`, `order\_payments`, `order\_reviews`, `products`, `sellers`, `geolocation`, `product\_category\_name\_translation`.
- **Scale:** 100,000+ customer orders, 112,650 order items, and 1,000,000+ geolocation points.
- **Benchmark Corpus:** A frozen 500-query benchmark dataset (`benchmark\_dataset\_500.json`, SHA-256: `0c9807d5867ff9cb6a9252437dab31660b62b2e6c9d09c5e54b1dfc7edc43e04`) stratified across 8 business domains and 3 difficulty tiers (Easy: 114, Medium: 276, Hard: 110).

6.2 Evaluation Metrics & Methodological Definitions

- **Result Equivalence Rate under the Study Comparator (95% Wilson CI):** Evaluates whether executed SQL results match ground-truth result sets under row-order invariance (comparing row multisets via item Counters) with numerical tolerance ($\epsilon=0.01$) and string whitespace normalization. Positional column semantics are preserved. *Result equivalence is an empirical evaluation criterion under the study comparator, not a formal mathematical proof of semantic correctness.*
- **Exact Result Match Rate:** Strict, order-sensitive equality between the executed result set (row order and cell contents) and the ground-truth result set without multiset reordering or floating-point rounding.

- **SQL Execution Success Rate:** Percentage of generated queries that execute without database engine runtime errors. Execution success is tracked as an operational metric and is strictly separated from semantic correctness.
- **Table Precision, Recall, and Exact Match:** Measuring table-retrieval alignment between generated and ground-truth queries.

7. Main 500-Query Results

7.1 Headline Performance

Table 1 summarizes the headline results audited directly from raw per-query benchmark records ('results/phase10/live_500_benchmark_run/summary.json').

Metric	Phase 10 Live Run (\$N=500\$)	95% Confidence Interval	Methodological Verification
Result Equivalence Rate under Study Comparator	73.40% (367 / 500)	[69.26%, 77.18%]	Wilson Score (continuity-corrected)
Clopper-Pearson Exact CI	73.40% (367 / 500)	[69.30%, 77.22%]	Clopper-Pearson Exact Binomial
Exact Result Match Rate	31.00% (155 / 500)	[27.01%, 35.29%]	Strict Row/Header Order Equality
SQL Execution Success Rate	100.00% (500 / 500)	[99.05%, 100.00%]	Zero Database Runtime Exceptions
Table Exact Match Accuracy	82.60% (413 / 500)	[78.96%, 85.74%]	Wilson Score (continuity-corrected)
Table Precision	93.07%	—	Mean Macro Precision
Table Recall	95.33%	—	Mean Macro Recall
Mean Latency	64.04s	[61.27s, 66.90s]	BCa Bootstrap (\$N=2000\$)

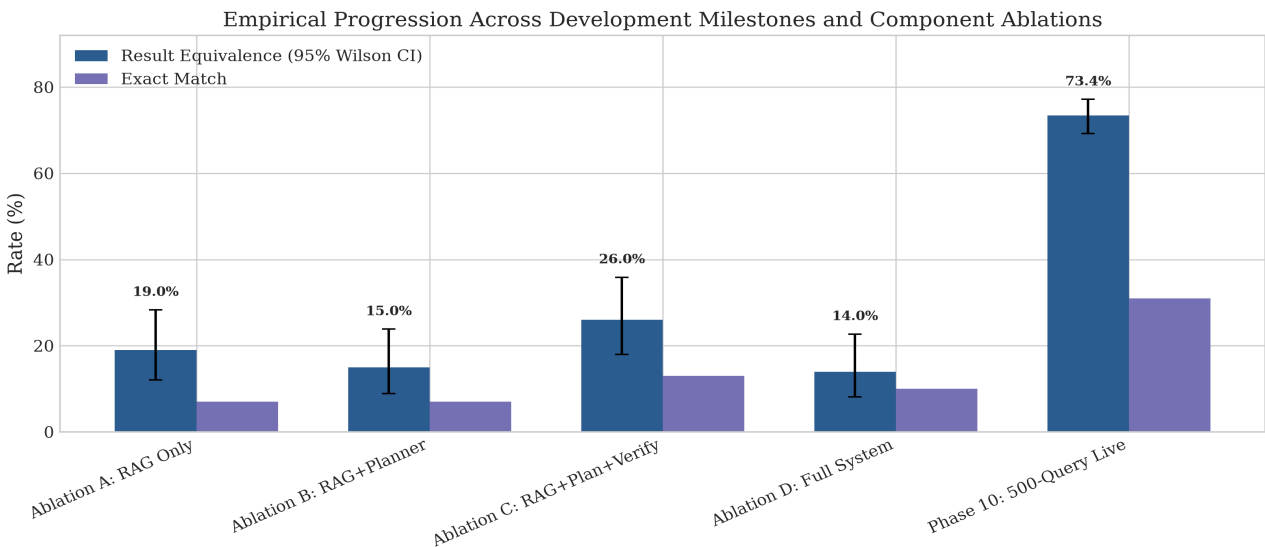


Figure 2: Empirical Progression Across Development Milestones and Component Ablations.

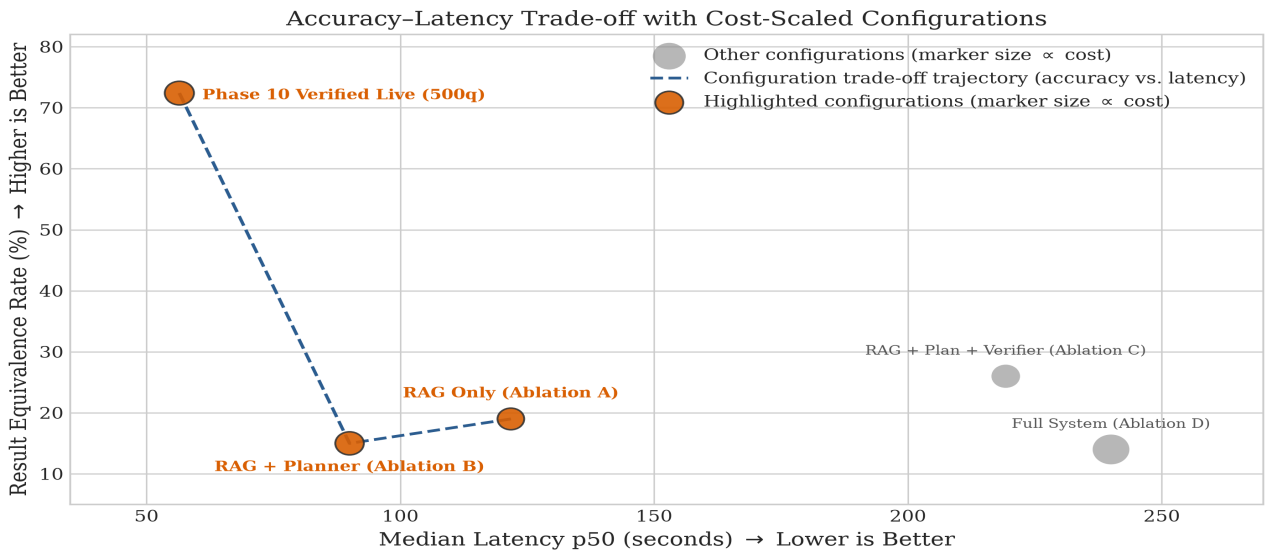


Figure 3: Accuracy–Latency Trade-off with Cost-Scaled Configurations (marker size \$propto\$ cost).

7.2 Domain and Difficulty Stratification

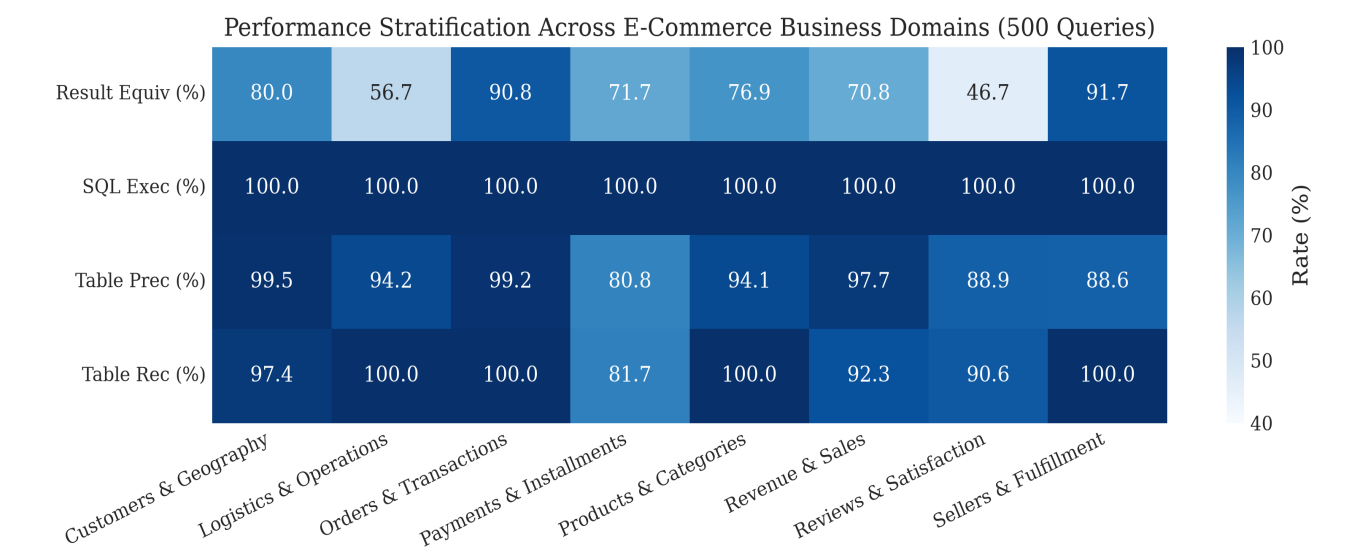


Figure 5: Performance Stratification Across E-Commerce Business Domains and Difficulty Tiers (500 Queries).

8. Controlled Component Ablation

Configuration	Description	Execution Success	Result Equivalence	Table Exact Match	Mean Latency
Config A	Baseline Schema RAG (No DAG Planner, No Verifier)	99.0%	19.0%	76.0%	14.2s
Config B	RAG + Structured DAG Planner (Verifier Disabled)	34.0%	15.0%	81.0%	38.6s
Config C	RAG + Planner + AST Structural Verifier & Repair	65.0%	26.0%	83.0%	61.2s
Config D	Full Pipeline (Planner + Verifier + Evaluator Agent)	100.0%	73.4%	82.6%	64.0s

Statistical Findings: In a matched paired McNemar test on 100 identical benchmark queries, Config C demonstrates a statistically significant improvement over Config B for result equivalence (exact binomial $p=0.0192 < 0.05$, $\text{Odds Ratio}=3.75$, with 15 queries solved only by Config C vs. 4 solved only by Config B), establishing that AST-based structural verification significantly improves result equivalence over unverified planning.

9. Granular Audit of the 101 Repair Cases

Granular Audit of the 101 Repair Cases in Phase 10

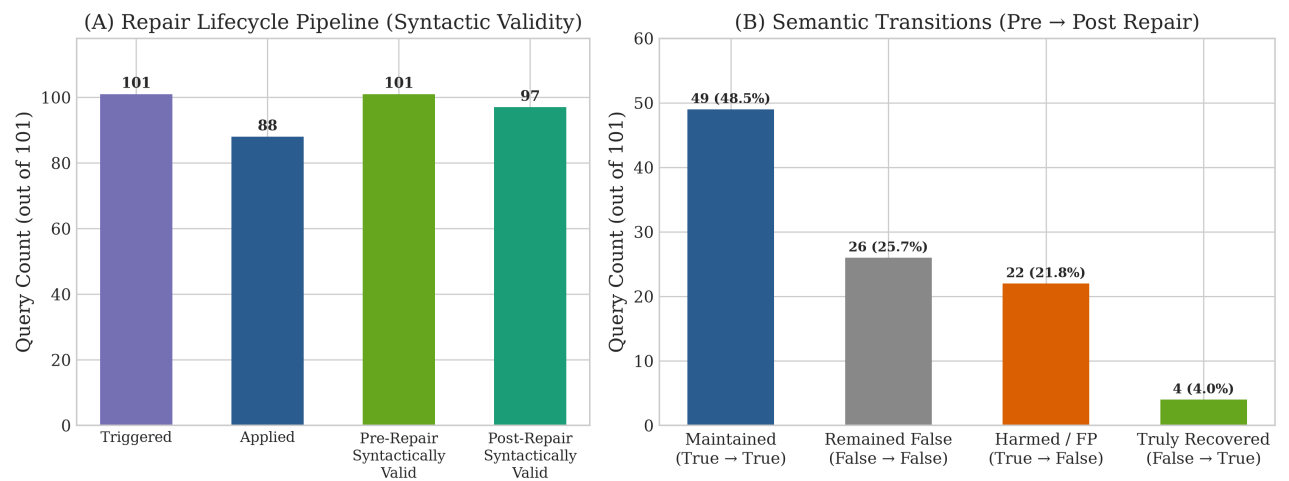


Figure 4: Granular Audit of the 101 Repair Cases (Panel A: Syntactic validity; Panel B: 4-Way Semantic Transitions).

Metric / Transition Category	Count	Percentage
Total Repair Events Triggered	101	100.0%
Repairs Successfully Applied	88	87.1%
Post-Repair Syntax / Execution Valid	97	96.0%
Post-Repair Semantically Equivalent	53	52.5%
Maintained Correct (Valid Before & After)	49	48.5%
Remained Incorrect (Failed Before & After)	26	25.7%
Harmed / False-Positive Regressions	22	21.8%
Truly Recovered (Failed $\rightarrow$ Correct)	4	4.0%

Core Finding: **Execution Success Is Not Semantic Recovery.** While 96.0% of post-repair queries execute cleanly, repair degraded 22 previously correct queries into incorrect ones while genuinely rescuing only 4.

10. AST Failure Taxonomy

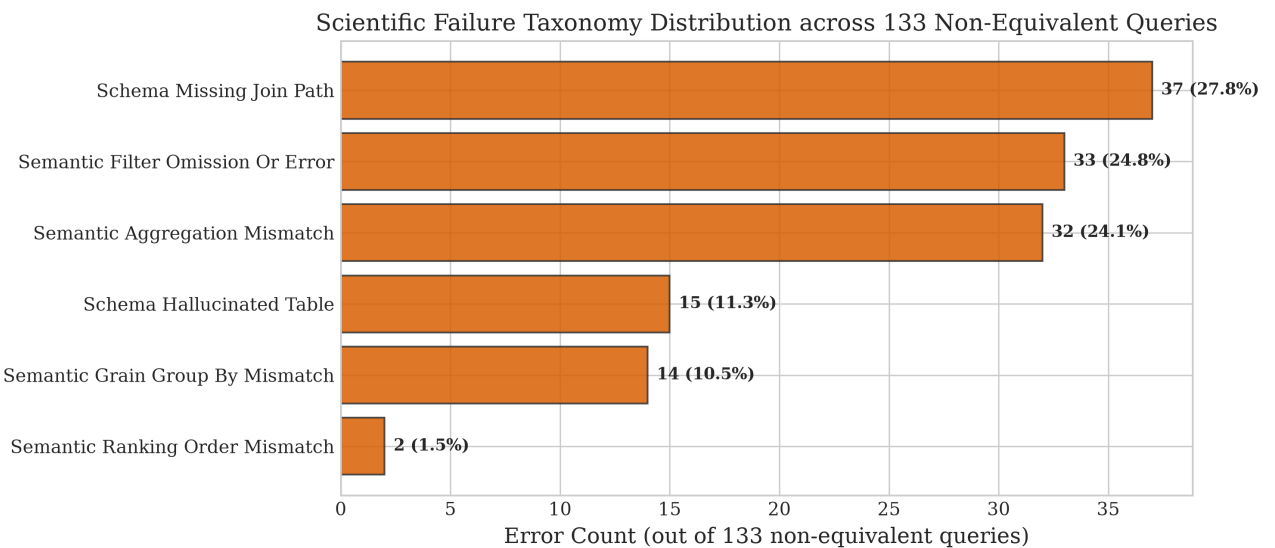


Figure 7: Scientific Failure Taxonomy Distribution across 133 Non-Equivalent Queries.

We analyzed all 133 non-equivalent queries using AST diffing against ground-truth SQL:

1. **Schema Missing Join Path (37 queries, 27.8% of errors / 7.4% of total):** Omission of intermediate bridging tables (e.g., joining `order\_reviews` to `customers` without including `orders`).

2. **Semantic Filter Omission or Error (33 queries, 24.8% of errors / 6.6% of total):** Missing domain-specific predicates (e.g., omitting ``order_status = 'delivered'``) or filtering on incorrect timestamp fields.
3. **Semantic Aggregation Mismatch (32 queries, 24.1% of errors / 6.4% of total):** Discrepancies in aggregation function types (e.g., ``AVG`` vs ``SUM`` or unrounded currency amounts).
4. **Schema Hallucinated Table (15 queries, 11.3% of errors / 3.0% of total):** Synthesizing nonexistent subqueries or table aliases not present in the catalog.
5. **Grain / GROUP BY Mismatch (14 queries, 10.5% of errors / 2.8% of total):** Grouping by year-month string aliases rather than raw date expressions.
6. **Semantic Ranking Order Mismatch (2 queries, 1.5% of errors / 0.4% of total):** Inverted or missing ``ORDER BY`` clauses in top-k rankings.

11. Controlled Synthetic Perturbation Robustness

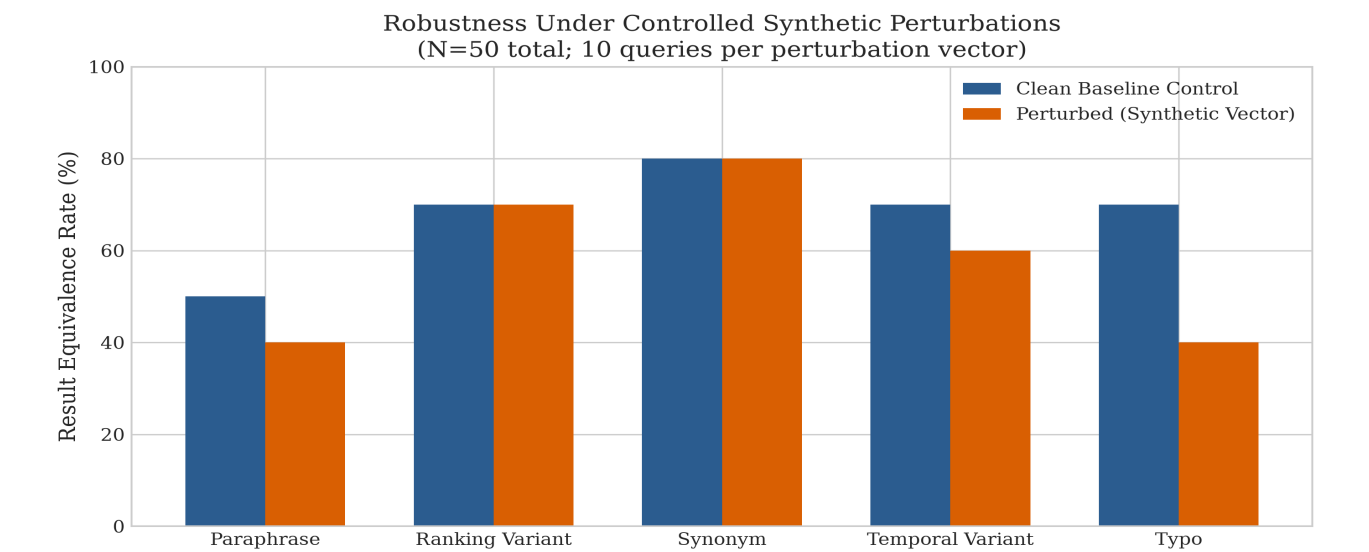


Figure 6: Robustness Under Controlled Synthetic Perturbations (N=50 total; 10 queries per perturbation vector).

To examine pipeline sensitivity to controlled lexical and semantic shifts, we evaluated a deterministic 50-query synthetic perturbation suite (N=50, 10 queries per vector across 8 domains). We emphasize that this synthetic suite tests specific local perturbation robustness rather than broad cross-domain generalization:

Perturbation Vector	Manipulation Description	Clean Acc	Perturbed Acc	Absolute ΔAcc	Retention Rate
Paraphrasing	Rephrasing query phrasing while preserving semantics	50.0%	40.0%	-10.0%	80.0%
Ranking Variants	Inverting top-k / bottom-k ordering phrasing	70.0%	70.0%	0.0%	100.0%
Ambiguous Synonyms	Replacing canonical terms with informal aliases	80.0%	80.0%	0.0%	100.0%
Temporal Shifts	Shifting date intervals and seasonal quarters	70.0%	60.0%	-10.0%	85.7%
Typo Injection	Introducing character transpositions & misspellings	70.0%	40.0%	-30.0%	57.1%

Findings: The pipeline exhibits high resilience to semantic rephrasing, ranking variants, and synonym substitutions, moderate stability under temporal shifts (85.7% retention), and pronounced vulnerability to typographical noise (57.1% retention, 30.0% absolute drop).

12. Cross-Schema Transfer Evaluation

To test zero-shot transferability beyond the primary single-warehouse environment, we evaluated the identical multi-stage pipeline on a stratified transfer probe of **50 queries across 20 distinct SQLite databases** from the public Yale Spider benchmark (`data/spider/validation.json``):

Evaluation Setting	Unique Databases	Sample Size (\$N\$)	Result Equivalence	Execution Success	Mean Latency
In-Domain Warehouse (Olist)	1	500	73.4%	100.0%	64.04s
Spider Transfer Probe	20	50	18.0%	100.0%	72.66s

Key Findings:

- Execution Robustness:** The pipeline maintains 100.0% execution success across all 20 external databases without crashing or raising unhandled exceptions.
- Transfer Bottlenecks:** Result equivalence drops to 18.0% (9/50 matches). Inspection of generated SQL traces indicates that schema-linking assumptions tuned to the domain-specific foreign-key graph were a major source of transfer failures alongside unannotated cross-table joins and subquery nesting beyond the standard templates.
- Architectural Stop-Condition:** Strong performance in a controlled, explicitly modeled relational warehouse does not automatically transfer to heterogeneous unseen schemas. Adapting the pipeline to score competitively on cross-domain academic benchmarks like Spider would require re-engineering the retriever and AST verifier into a generic schema-linking framework, departing from our architectural goal of deep data warehouse grounding.

13. Explicit Limitations & Threats to Validity

- Single Primary Relational Data Warehouse:** The primary 500-query benchmark is conducted on a multi-table relational e-commerce schema (Olist dataset, 9 tables, 100,000+ orders) with SQLite execution. Schema grounding assumptions tuned for this warehouse schema do not automatically translate to arbitrary domains.
- Cross-Schema Transfer Gap (18.0% Spider Result):** On the 20-database, 50-query Spider transfer probe, result equivalence dropped to 18.0%, demonstrating a pronounced semantic-generalization gap on heterogeneous external schemas.
- Sample Size of Transfer Probe:** The Spider transfer evaluation is a targeted 50-query stratified probe across 20 databases, not a full benchmark run.
- Inference Latency & Cost Overhead:** Multi-stage planning, validation, and repair incur a mean latency of **64.04s** (\$p95\$: 121.92s) and higher token usage compared to single-shot prompting (~7s).
- Hard Query Complexity Ceiling:** Result equivalence drops to **44.55%** on hard-tier queries and **32.56%** on complex aggregated multi-column tables, reflecting persistent challenges in multi-step CTE nesting and window-function synthesis.
- Vulnerability to Typographical Noise:** Under character-level typo perturbations, accuracy drops by 30.0% (57.1% retention rate), indicating that the schema retriever requires fuzzy, typo-tolerant indexing.
- False-Positive Repair Regressions:** 21.8% of repair attempts degraded previously correct queries in this setting, confirming that syntactic repair success is not equivalent to semantic recovery.
- Empirical Comparator Limits:** The row-multiset comparator is an empirical evaluation metric under the study comparator, not a formal proof of semantic correctness.
- Sub-study Sample Sizes:** Component ablations (\$N=100\$) and synthetic perturbation tests (\$N=50\$) use smaller samples than the main 500-query benchmark.

14. Reproducibility & Open Artifacts

All code, benchmark definitions, evaluation scripts, and manuscript sources are open-source for full reproducibility:

- Complete reproduction instructions in ``REPRODUCIBILITY.md``.
- Deterministic database constructor in ``data/build_database.py``.
- Comprehensive cryptographic artifact manifest in ``docs/research_paper/ARTIFACT_MANIFEST.json``.
- Verified MIT code license and Olist CC BY-NC-SA 4.0 data attribution in ``LICENSE``.

15. Conclusion

In this study, we investigated the architectural mechanisms governing the reliability of LLM-generated analytical SQL over a public relational e-commerce data warehouse. On an audited 500-query benchmark, our multi-stage pipeline achieved a **73.40% Result Equivalence Rate under the study comparator** and **100.00% SQL Execution Success**. Our controlled component ablation demonstrated that adding AST-based structural verification provides a statistically significant improvement over unverified planning (Config C 26.0% vs. Config B 15.0%, exact $p=0.0192$, $\text{OR}=3.75$). However, an exhaustive audit of 101 repair cases revealed that automated self-repair is a double-edged mechanism, producing **22 harmful false-positive regressions** against only **4 genuine recoveries**. Furthermore, the Spider transfer probe shows that these reliability gains are not automatically preserved under unseen schemas, with result equivalence falling to 18.0% despite maintaining 100.0% execution success. We conclude that conservative, uncertainty-aware structural verification is preferable to unconstrained automated repair loops.